

MRAM Subsystem — IP Specification

Sep 24, 2026 · @Avi

Overview

This subsystem gives the RTPF500TCG1509 fabric access to 128MB of Avalanche AS302G208 MRAM over a single Quad-SPI interface, presented to the AXI interconnect as one AXI4 slave target shared by two masters: the NOEL-V (RISC-V) CPU and a PCI master.

- MRAM connects via one Quad-SPI channel (CS#, SCLK, IO0-3, WP#).
- Both AXI masters use identity addressing across the full 128MB range -- AXI address equals MRAM address.
- Address 0 always holds a working copy of the boot image, refreshed from a password-protected master copy at the top of the address space on every reset. NOEL-V's fixed reset vector points here.
- No cache sits in the data path: the workload -- small, largely random accesses to a metadata table -- does not benefit from one.

This document covers the architecture, address map, access control, and the VHDL modules and state machines that implement it.

Requirements & Constraints

Item	Value
Target device	RTPF500TCG1509 (RT PolarFire)
MRAM device	Avalanche AS302G208, 2Gb, Quad-SPI
Usable MRAM capacity	128MB over one Quad-SPI channel
MRAM interface	CS#, SCLK, IO0-3, WP#
AXI masters	RISC-V (NOEL-V) and PCI, arbitrated by the AXI interconnect into one slave port
AXI clock	150MHz

Item	Value
Access control	Both masters read/write general memory freely; writes to the 32MB boot region require an externally-derived <code>key_ok</code> unlock signal
Software	NOEL-V (RISC-V) soft core in the RT PolarFire fabric

Design priorities, highest first:

1. Reuse the existing PF_SRAM_AHB_AXI interface
2. Minimize FPGA resources
3. Preserve software compatibility (no memory-map changes)
4. Maximize sustained MRAM throughput
5. Minimize miss latency
6. Avoid unnecessary complexity

Top-Level Architecture

flowchart TD

```

M1[RISC-V / NOEL-V AXI Master] --> IC[AXI Interconnect]
M2[PCI AXI Master] --> IC
IC -->|one arbitrated AXI4 slave port| C[AXI4 Slave Wrapper]
BC[MRAM Boot Copy] -->|mux, selected by boot_done| D[MRAM Write Guard]
C -->|mux, selected by boot_done| D
BC -->|cpu_reset_n| CPU[NOEL-V CPU Reset Input]
PW[External unlock comparison] -. ->|key_ok| D
D -->|core_req/core_resp| E[MRAM QSPI Backend]
E -->|CS#, SCLK, I00-3| G[AS302G208 MRAM, 128MB]

```

- **AXI Interconnect** (outside this design's files): arbitrates RISC-V and PCI into a single AXI4 slave port with identity addressing.
- **AXI4 Slave Wrapper** (`axi4_slave_wrapper.vhd`): translates AXI4 bursts into single-beat requests on a common `core_req / core_resp` interface (`mram_pkg.vhd`).
- **MRAM Boot Copy** (`mram_boot_copy.vhd`): at reset, holds the CPU in reset and copies the 32MB master boot image to address 0, verifying it before releasing the CPU.
- **MRAM Write Guard** (`mram_write_guard.vhd`): passes reads through unconditionally; gates writes to the protected region on a password match.

- **MRAM QSPI Backend** (`mram_qspi_backend.vhd`): drives the physical QSPI pins directly, one request at a time, and owns the four pins permanently.

Address Map & Access Control

A 27-bit byte address covers the 128MB range with identity addressing for both masters.

Region	MRAM address range	RISC-V	PCI
Working copy / CPU boot & general RAM	0x0 - 0x1FFFFFFF (32MB)	Read/write: yes -- NOEL-V's fixed reset vector points here	Read/write: yes
General RAM (continued)	0x20000000 - 0x5FFFFFFF (64MB)	Read/write: yes	Read/write: yes
Master boot copy (protected)	0x60000000 - 0x7FFFFFFF (32MB)	Read: yes. Write: only with a matching password	Read: yes. Write: only with a matching password

NOEL-V boots from address 0, which `mram_boot_copy` refreshes from the master copy on every reset. The AXI interconnect's target window for this slave should be configured as 128MB.

`mram_write_guard.vhd` sits between the AXI wrapper (and boot copy) and the MRAM backend. Its only role is the write check below.

Access Control -- MRAM Write Guard

flowchart TD

```

    A[core_req arrives] --> B{Write AND address in protected region?}
    B -->|No| C[Pass through to backend unchanged]
    B -->|Yes| D{key_ok = 1?}
    D -->|Yes| C
    D -->|No| E[Accept immediately, answer SLVERR one cycle later, drop the request]
  
```

- Reads: always pass through, any address.

- Writes outside the protected region: always pass through.
- Writes inside it: pass only while `key_ok` -- a single-bit input -- is high. The comparison behind it (password check or otherwise) is performed entirely outside this design; the write guard trusts it completely.
- Gates by address and password only, not by requesting master. In practice only PCI can satisfy the check, since only PCI can write the password register, but the hardware itself does not distinguish masters by AXI ID.
- The write guard has no visibility into how `key_ok` is derived or how strong the underlying check is -- it trusts the signal completely. That comparison's strength (password width, timing, anything else) is entirely the external logic's responsibility, not this design's.

AXI4 Slave Wrapper

Presents the standard AXI4 slave signal set; converts AW/W/B and AR/R traffic into single-beat requests on `core_req / core_resp` (`mram_pkg.vhd`).

Write state machine:

```
stateDiagram-v2
    [*] --> WR_IDLE
    WR_IDLE --> WR_DATA: AWVALID, INCR & full-width
    WR_IDLE --> WR_ERR_DRAIN: AWVALID, unsupported burst/size
    WR_ERR_DRAIN --> WR_ERR_RESP: WLAST
    WR_ERR_RESP --> WR_IDLE: BREADY
    WR_DATA --> WR_WAIT_BVALID: core accepts beat
    WR_WAIT_BVALID --> WR_DATA: bvalid, beats remain
    WR_WAIT_BVALID --> WR_RESP: bvalid, last beat
    WR_RESP --> WR_IDLE: BREADY
```

Read state machine (per outstanding-FIFO entry, up to 4 bursts deep):

```
stateDiagram-v2
    [*] --> RD_IDLE
    RD_IDLE --> RD_OUTPUT: FIFO head is an error entry
    RD_IDLE --> RD_ISSUE: FIFO head is valid
    RD_ISSUE --> RD_WAIT_RVALID: core accepts read
    RD_WAIT_RVALID --> RD_OUTPUT: rvalid
    RD_OUTPUT --> RD_ISSUE: RREADY, beats remain
    RD_OUTPUT --> RD_IDLE: RREADY, last beat
```

Documented scope, not oversights:

- INCR bursts only; FIXED/WRAP get SLVERR.
- Full 512-bit (64-byte) beats only; a narrower AWSIZE/ARSIZE gets SLVERR.
- Beats served strictly in order, one at a time -- no write or read pipelining within a burst.
- Fixed write-over-read priority for the shared `core_req` interface.
- In-order completion assumed between wrapper and MRAM backend.

The read-side FIFO push (AR-accept) and pop (R-drain) are handled in a single process, so the FIFO state has one driver.

MRAM Boot Copy

```
stateDiagram-v2
    [*] --> COPY_READ
    COPY_READ --> COPY_WRITE: chunk read
    COPY_WRITE --> COPY_READ: chunk written, more remain
    COPY_WRITE --> VERIFY_READ_SRC: chunk written, copy complete
    VERIFY_READ_SRC --> VERIFY_READ_DST: source chunk read
    VERIFY_READ_DST --> VERIFY_COMPARE: destination chunk read
    VERIFY_COMPARE --> VERIFY_READ_SRC: match, more remain
    VERIFY_COMPARE --> DONE: match, verify complete
    VERIFY_COMPARE --> RETRY: mismatch
    RETRY --> COPY_READ: retries remain
    RETRY --> FAIL: retries exhausted
    DONE --> [*]
    FAIL --> [*]
```

- Copies the 32MB master image (0x6000000) to address 0, 64 bytes at a time, entirely through `mram_qspi_backend` -- MRAM talking to itself, no AXI interconnect involved.
- Re-reads both sides of every chunk and compares before trusting the copy.
- A mismatch retries the whole copy, up to 3 times. A watchdog forces a retry if the FSM stalls for an extended period with no progress.
- If all retries are exhausted, `boot_fail` asserts and `cpu_reset_n` stays low permanently -- the CPU never runs from unverified data.
- Timing at the actual 150MHz AXI clock, default SCLK divider (37.5MHz SCLK): ~8.64s for one copy+verify pass, ~34.6s worst case with all retries exhausted -- both well inside a one-minute boot budget.

MRAM Backend & QSPI Master

Drives the AS302G208 directly per request, one at a time -- no cache, no line concept. A request for N bytes moves exactly N bytes.

Framing is (1-4-4): opcode on IO0 only (single-bit), address and data on all four IOs, SDR. Opcodes, per the datasheet's instruction table: Read Quad I/O (RDQI) EBh; Write Quad I/O (4WQIO) D2h, preceded by its own complete Write Enable (WREN, 06h) transaction. Narrow requests are supported directly -- the byte lane implied by the low address bits is extracted (reads) or positioned (writes) via a barrel shift against the request's `size` field.

`G_DUMMY_CYCLES` (default 8) is a placeholder: the datasheet specifies these latency cycles as configurable via Configuration Register 2 (CR2[3:0]) and frequency-dependent, not fixed. This design assumes CR2 is configured elsewhere to match; configuring CR2 itself is out of scope here.

Not implemented: burst aggregation/prefetch across consecutive requests, XIP mode.

Reliability

Single-bit SPI's SI is always an FPGA output and SO always an FPGA input -- no shared line, so pin-level bus contention is structurally impossible. Quad I/O's four lines are bidirectional and tri-stated, switching direction by phase; a corrupted output-enable control at the wrong moment, or a timing mismatch, can cause contention. The AS302G208 silicon itself is qualified for quad operation in this environment; the relevant risk is in the FPGA-side control logic around it.

Mitigation in this RTL: both `mram_boot_copy` and `mram_qspi_backend` force the bus released and CS# deasserted if their state machine ever lands on an unreachable encoding, rather than continuing from a corrupted state. Two further mitigations are the synthesis tool's responsibility, not expressible in this RTL: enabling illegal-state-recovery ("safe FSM") support for these FSMs, and applying TMR if that is standard practice elsewhere in this design.

Workload Characterization

From the software team's code-reading estimate (not yet measured):

NAND operation	Memory access pattern
Per 16KB NAND read	One random 4-byte memory read
Per 16KB NAND write	Two random 4-byte memory writes
Occasional (region switch, cleanup)	~32KB sequential MRAM read or write, ~1ms at ~40MB/s

This looks like flash-translation-layer metadata traffic (logical-to-physical mapping lookups per NAND page, with occasional bulk region rewrites for garbage collection). The dominant traffic is tiny (4B) and largely random, which is why this design moves exactly the requested size per transaction rather than a fixed block -- see Bandwidth & Performance Analysis. The occasional ~32KB sequential operation is naturally efficient here, and the ~40MB/s estimate is consistent with the channel bandwidth below.

Bandwidth & Performance Analysis

Single Quad-SPI channel peak: ~50-54MB/s raw bit rate (108MHz SDR / 54MHz DDR non-derated; 100MHz SDR / 50MHz DDR on the radiation-derated grade -- the same peak either way, since DDR halves the clock and doubles the per-clock transfer).

Per-transaction overhead model (quad SDR, 4-byte addressing, ~8 dummy cycles assumed): a 4-byte request costs ~16 overhead cycles + 8 data cycles -- about 0.24us at 100MHz, moving exactly the 4 bytes needed. A fixed 64-byte transaction for the same 4-byte need would cost ~1.44us and move 16x the necessary data. This is why the backend moves exactly the requested size rather than a fixed block.

Resource Utilization Estimate

This design's footprint is dominated by modest control logic -- the AXI wrapper's FSMs, the QSPI backend's FSM and shift register, the write guard's combinational check, the boot-copy sequencer's FSM -- with no significant LSRAM usage.

RTPF500T's budget, for reference: ~1,520 LSRAM blocks (20Kbit each), ~4,440 uSRAM blocks (768-bit each), ~481K logic elements.

No grounded LUT/FF count exists yet -- that needs an actual synthesis run.

Open Items

- `G_DUMMY_CYCLES` (assumed 8) is not confirmed against the datasheet's CR2 configuration table.
- Whether `key_ok` needs synchronization at this boundary -- if it originates outside the `ac1k` domain, it should be double-registered before use; this design assumes it already arrives synchronous to `ac1k`.
- Whether strictly PCI-only write access (regardless of register contents) is required, which would need the requesting master's AXI ID threaded through from the wrapper - - not implemented in this revision.
- No VHDL toolchain was available while authoring this design; none of it has been compiled or simulated. Treat it as a structurally-reasoned design pending verification.